

=====Array Methods=====

Arrays Methods

```
sort()
binarySearch()
equals()
fill()
copyOf()
copyOfRange()
toString()
asList()
parallelSort()
setAll()
stream()
```

The Arrays class in Java is a part of the java.util package and provides various static methods for manipulating arrays.

It includes methods for sorting, searching, comparing, filling, and converting arrays.

=>Arrays class contains static methods only.

=>It provides utility methods for array operations.

=>Methods can perform operations such as sorting, searching, filling, copying, and converting arrays.

=>It includes methods for parallel sorting and working with streams.

Arrays Methods:

sort():

The sort() method sorts the specified array into ascending order.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {5, 3, 8, 1, 2};
        Arrays.sort(numbers);
        System.out.println(Arrays.toString(numbers));
    }
}
```

binarySearch():

The `binarySearch()` method searches the specified array for the specified value using the binary search algorithm.

The array must be sorted before calling this method.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 5, 8};
        int index = Arrays.binarySearch(numbers, 5);
        System.out.println("Index of 5: " + index);
    }
}
```

equals():

The `equals()` method returns true if the two specified arrays are equal to one another.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers1 = {1, 2, 3, 5, 8};
        int[] numbers2 = {1, 2, 3, 5, 8};
        boolean isEqual = Arrays.equals(numbers1, numbers2);
        System.out.println("Arrays are equal: " + isEqual);
    }
}
```

fill():

The `fill()` method assigns the specified value to each element of the specified array.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = new int[5];
        Arrays.fill(numbers, 9);
        System.out.println(Arrays.toString(numbers));
    }
}
```

copyOf():

The copyOf() method copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        int[] newNumbers = Arrays.copyOf(numbers, 3);
        System.out.println(Arrays.toString(newNumbers));
    }
}
```

copyOfRange():

The copyOfRange() method copies the specified range of the specified array into a new array.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        int[] newNumbers = Arrays.copyOfRange(numbers, 1, 4);
        System.out.println(Arrays.toString(newNumbers));
    }
}
```

toString():

The toString() method returns a string representation of the contents of the specified array.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        String arrayString = Arrays.toString(numbers);
        System.out.println(arrayString);
    }
}
asList():
```

The `asList()` method returns a fixed-size list backed by the specified array.

Example:

```
import java.util.Arrays;
import java.util.List;
public class ArraysExample {
    public static void main(String[] args) {
        String[] fruits = {"Apple", "Mango", "Banana"};
        List<String> fruitList = Arrays.asList(fruits);
        System.out.println(fruitList);
    }
}
```

parallelSort():

The `parallelSort()` method sorts the specified array into ascending numerical order using the specified parallel sorting algorithm

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {5, 3, 8, 1, 2};
        Arrays.parallelSort(numbers);
        System.out.println(Arrays.toString(numbers));
    }
}
setAll():
```

The `setAll()` method sets all elements of the specified array, using the provided generator function to compute each element.

Example:

```
import java.util.Arrays;
import java.util.function.IntUnaryOperator;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = new int[5];
        Arrays.setAll(numbers, i -> i + 1);
        System.out.println(Arrays.toString(numbers));
    }
}
```

stream():

The stream() method returns a sequential Stream with the specified array as its source.

Example:

```
import java.util.Arrays;
public class ArraysExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        Arrays.stream(numbers).forEach(System.out::println);
    }
}
```